

# Modul 1 Moderationsnotizen

Vorbereitung und Referenz während des Trainings

**Stand:** 17.09.2026. Bezogen auf die überarbeitete Präsentation mit 48 Folien und 120 Minuten einschließlich Pause. Die Foliennummern zählen ab der Titelfolie und entsprechen den PDF-Seiten. Bildfolien zeigen teilweise keine sichtbare Nummer.

## Ablauf auf einen Blick

| Minute  | Folien | Inhalt                         | Notizen ab Seite |
|---------|--------|--------------------------------|------------------|
| 00-10   | 1-5    | Einstieg und Erfahrungen       | 2                |
| 10-32   | 6-16   | Modell und Generierung         | 2-4              |
| 32-48   | 17-23  | Fehleranalyse und Verifikation | 5-6              |
| 48-60   | 24-30  | Kontext und sichere Nutzung    | 7                |
| 60-70   | 31     | Pause                          | 7                |
| 70-88   | 32-35  | Kontextvergleich               | 8                |
| 88-108  | 36-44  | RAG, Tools und Agenten         | 9-10             |
| 108-120 | 45-48  | Wissenscheck und Transfer      | 11               |

**Zeitanker:** Übung 1 beginnt etwa bei Minute 22. Fallanalyse beginnt bei Minute 34. Pause startet bei Minute 60. Nach der Pause bei Minute 70 fortsetzen. Architekturübung startet bei Minute 96. Abschluss beginnt bei Minute 108.

## Vorbereitung vor dem Termin

- PDF oder HTML öffnen. Bei HTML den Ordner assets mitnehmen und Bildanzeige prüfen.
- Das Arbeitsblatt arbeitsblatt-modul-1.md vorab verteilen. Prompts müssen kopierbar sein.
- Einen freigegebenen Modellzugang pro Pair prüfen. Nur synthetische Daten verwenden.
- Timer und gemeinsame Sammelfläche für „Hilft / Nervt“ vorbereiten.
- Kein Projektsetup voraussetzen. Die Bestellübung ist ein Review und kein Testlauf.
- Diese Referenz ausdrucken. Auf Seite 12 stehen Antworten für häufige Rückfragen.

## Vorgehen während des Trainings

**Sprechimpulse sind Formulierungshilfen.** Nicht vorlesen müssen. Erst nachfragen, dann erklären. Nach jeder Übung eine beobachtete Entscheidung mit dem zugrunde liegenden Mechanismus verbinden.

**Bei Zeitdruck:** Zusätzliche Wortmeldungen und technische Vertiefungen kürzen. Fragen sichtbar parken. Pause und Pair-Arbeit schützen. Beim Review auf die sechs priorisierten Fälle beschränken. Pro Gruppe nur einen strittigen Architekturpunkt besprechen.

**Ohne Modellzugang:** Fallanalyse unverändert durchführen. In der Bestellübung zunächst Regeln und Pseudocode aus dem Basisprompt ableiten, danach mit den Fachregeln überarbeiten. Kein Live-Ergebnis ist Voraussetzung für den Lernerfolg.

**Eigene Angaben:** Startzeit \_\_\_\_\_ Pause \_\_\_\_\_ Ende \_\_\_\_\_

# Einstieg und Lernziel

Minute 00-10 | Folien 1-5 | Übergang zu Folie 6

## Folie 1 AI Basics for Developers

**Sprechimpuls:** „Heute bauen wir das Verständnis auf, das uns in den nächsten Modulen hilft: Was kann ein Modell beitragen, welche Informationen braucht es, und woran erkennen wir ein brauchbares Ergebnis?“

**Rahmen setzen:** Grundlegende Programmiererfahrung genügt. Keine ML-Vorkenntnisse nötig. Die Beispiele verwenden überwiegend C# und SQL, ein Analysefall Python. Die Konzepte gelten unabhängig von der Sprache.

## Folie 2 Nach diesem Modul könnt ihr

**Betonen:** Modell und Coding Assistant unterscheiden, Generierung und Kontext erklären sowie passende Prüfungen auswählen. Begriffe wie RAG und Agenten sollen die Teilnehmer einordnen können. Eigene Agenten implementieren sie erst in Modul 6.

## Folie 3 Unser Ablauf

**Moderation:** Einstieg, Lernziele und Agenda zusammen in drei Minuten. Pause nach einer Stunde ankündigen. Auf Übungen und gemeinsame Auswertung hinweisen. Die angegebenen Zeiten enthalten Übungen und Übergänge; sie kommen nicht zusätzlich dazu.

## Folie 4 AI in eurem Entwickleralltag

**Auftrag vorlesen:** „Tauscht euch drei Minuten zu zweit aus. Wo hilft euch AI, wo kostet sie Zeit, und was möchtet ihr heute besser verstehen? Wählt danach ein Beispiel für unsere Sammlung.“ Timer starten.

**Wenn jemand noch keine Erfahrung hat:** Nach Erwartungen oder einer beobachteten Nutzung fragen. Die Person kann mit einer erfahreneren Person ein Pair bilden. Niemand muss einen eigenen Modellzugang vorführen.

## Folie 5 Unsere Erfahrungen als Ausgangspunkt

**Vier Minuten sammeln:** Beispiele unter „Hilft“ und „Nervt“ notieren. Bei großen Gruppen vier Beiträge hören, den Rest schriftlich sammeln. Noch nicht jeden Fall erklären oder korrigieren.

**Nachfragen:** „Woran habt ihr erkannt, dass das Ergebnis gut war?“ oder „Welche Information fehlte?“ Ein Beispiel zu Code, Tests oder erfundenen APIs für spätere Rückbezüge markieren.

**Erwartete Vielfalt:** Testentwürfe, Erklärungen, Routinecode, falsche APIs, unpassende Änderungen. Unterschiedliche Erfahrungen stehen lassen; sie sind Ausgangsmaterial und kein Modellvergleich.

## Folie 6 Wie ein LLM arbeitet

**Übergang bei Minute 10:** „Wir haben hilfreiche und frustrierende Ergebnisse gesammelt. Jetzt schauen wir darauf, wie diese Ausgaben entstehen und welche Rolle die Anwendung darum herum spielt.“

**Zeitdisziplin:** Für Folien 6-14 stehen zwölf Minuten einschließlich kurzer Verständnisfragen zur Verfügung. Transformer-Details und Produktvergleiche auf die Fragenliste verschieben.

**Eigene Beobachtungen aus dem Einstieg:**

---

---

# Modell und Anwendung unterscheiden

Minute 10-22 | Folien 7-10 | Erster Teil des Grundlagenimpulses

## Folie 7 AI und verwandte Begriffe

**Etwa 30 Sekunden:** „KI ist der Oberbegriff. Machine Learning lernt Muster aus Daten. Generative Modelle erzeugen Inhalte. Hier konzentrieren wir uns auf Sprachmodelle und ihren Einsatz beim Entwickeln.“

**Einordnung:** Die Grafik ist vereinfacht. Nicht jede KI ist ein LLM. Generative AI umfasst weitere Modelltypen. Eine Chatoberfläche kann Suche, Dateizugriff und Werkzeuge enthalten. Diese Fähigkeiten nicht automatisch dem Modell selbst zuschreiben.

## Folie 8 Was steckt in einem Coding Assistant

**Zwei Minuten:** Auf Auftrag, Anwendung und Modell zeigen. „Ihr stellt eine Aufgabe. Die Anwendung stellt Kontext zusammen. Das Modell erzeugt eine Antwort oder schlägt einen Tool-Aufruf vor. Die Anwendung entscheidet technisch, welche Aufrufe erlaubt sind, und führt sie aus.“

**Am Bild entlang erklären:** Dateien, Verlauf, gespeicherte Notizen und Tool-Ergebnisse können in den Kontext gelangen. Das Modell hat keinen automatischen Zugriff auf das gesamte Repository, das Internet oder eine Shell. Verfügbare Tools und Rechte hängen von der Anwendung ab.

**Prüffrage:** „Kennt der Assistant jede Datei, nur weil das Repository geöffnet ist?“ Erwartung: Das hängt davon ab, was die Anwendung auswählt und bereitstellt. Später bei Kontextfenster und RAG darauf zurückkommen.

## Folie 9 Training und Inference

**Eine Minute:** „Im Training werden Parameter angepasst. Bei einer normalen Anfrage nutzt das Modell diese gelernten Parameter und den aktuellen Kontext, um eine Ausgabe zu erzeugen.“

**Wichtige Präzisierung:** Eine zusätzliche Datei kann die Antwort beeinflussen, ohne die Parameter zu ändern. Das ist keine Aussage darüber, ob ein Anbieter Eingaben speichert oder später zum Training nutzt. Dafür gelten Produkt, Vertrag und Organisationsvorgaben.

**Übergang:** „Wie entstehen die Fähigkeiten, Anweisungen zu befolgen und komplexere Aufgaben zu bearbeiten?“

## Folie 10 Training Nachtraining und Reasoning

**Zwei Minuten:** Vortraining als Lernen aus vielen Beispielen erklären. Nachtraining formt unter anderem das Befolgen von Anweisungen und das Lösen von Aufgaben durch weitere Beispiele und Feedback.

**Reasoning einordnen:** Manche Modelle verwenden zusätzliche Rechenschritte bei einer Anfrage. Das kann bei schwierigen Aufgaben helfen, benötigt aber gegebenenfalls mehr Zeit und Rechenaufwand. Es bleibt eine Ausgabe, die geprüft werden muss.

**Nicht verkürzen auf:** „Das ist nur Wortassoziation.“ Die schrittweise Token-Erzeugung beschreibt den Ausgabemechanismus, nicht die Obergrenze möglicher Fähigkeiten. Angezeigte Denktexte sind kein verlässlicher vollständiger Einblick in die tatsächlichen Entscheidungsursachen.

**Übergang:** „Die Ausgabe entsteht in kleinen Einheiten. Diese Einheiten heißen Tokens.“

# Generierung und erste Übung

Minute 10-32 | Folien 11-16 | Impuls bis etwa Minute 22

## Folie 11 Tokens und Token IDs

**Eine Minute:** Tokens können Wörter, Wortteile oder Zeichen sein. Die IDs bezeichnen Einträge im Vokabular, keine Bedeutungswerte. Zerlegung und IDs im Bild sind erfunden; kein realer Tokenizer wurde damit gemessen.

## Folie 12 Eine plausible Fortsetzung als Text

**45 Sekunden:** „Für den nächsten Schritt ergibt sich eine Verteilung möglicher Tokens. Greedy wählt hier blau.“ Die Prozentwerte sind illustrative Werte. Wörter stehen vereinfacht für Tokens.

**Unbedingt sagen:** Die Wahrscheinlichkeit betrifft die Fortsetzung im Kontext, nicht die Wahrheit der Aussage über den Himmel. Greedy wählt das wahrscheinlichste nächste Token und garantiert nicht die wahrscheinlichste ganze Antwort.

## Folie 13 Eine plausible Fortsetzung als Code

**45 Sekunden:** Derselbe Mechanismus gilt für Code. Nach dem gewählten Token wird die Folge erweitert und die nächste Verteilung berechnet. `return` wirkt im Beispiel plausibel. Ob `return` oder `throw` fachlich passt, hängt von Anforderungen und umgebendem Code ab.

## Folie 14 Greedy und Sampling

**Eine Minute:** Greedy nimmt bei derselben Verteilung das größte Gewicht. Sampling zieht gewichtet aus der Verteilung. Die drei Ziehungen rechts sind unabhängige Beispiele für denselben Kontext, keine aufeinanderfolgenden Tokens einer Antwort.

**Merksatz:** Wiederholbarkeit beweist keine Wahrheit. Änderungen an Kontext, Modell oder Einstellungen können Ergebnisse zusätzlich beeinflussen.

## Folie 15 Be the LLM

**Auftrag eine Minute, dann sechs Minuten Pair-Arbeit:** „Findet zu jedem Fragment zwei plausible Fortsetzungen. Wählt eine bevorzugte Variante. Notiert, welche Information euch für eine fachliche Entscheidung fehlt.“ Papieraufgabe, kein Modellzugang nötig. SQL nicht ausführen.

**Während der Übung:** Nach drei Minuten an alle Fragmente erinnern. Nicht die „richtige“ Codezeile verraten. Bei Stillstand fragen: „Welche Annahme müsste stimmen, damit eure Variante sinnvoll wäre?“

## Folie 16 Auswertung Muster und Anforderungen

**Drei Minuten:** Zwei Pairs nach unterschiedlichen Fortsetzungen und fehlenden Informationen fragen. Paris führt gewöhnlich zu Frankreich, gleichnamige Orte zeigen die Rolle des Kontexts. `IsValid` benötigt Fachregeln und Fehlermodell. `WHERE` benötigt Schema, Dialekt und Suchziel.

**Leitfrage:** „Welche eurer Entscheidungen beruhte auf einer Annahme?“ Beispiele sind keine eindeutigen Musterlösungen. Die Begründung zählt.

**Übergang bei Minute 32:** „Ein Modell kann fehlende Information mit plausiblen Details füllen. Jetzt prüfen wir einen konkreten Entwurf darauf, was tatsächlich belegt ist.“

# Fallanalyse der internen Library

Minute 32-43 | Folien 17-20 | Lösung erst nach der Übung zeigen

## Folien 17 und 18 Auftrag und Material

**Zwei Minuten:** Abschnitt „Fehler erkennen und prüfen“ öffnen und Auftrag der Library-Folie vorlesen. Die Antwort ist als hypothetisch gekennzeichnet. Weitere Schnittstelleninformationen liegen nicht vor. Code ausschließlich analysieren, nicht ausführen.

**Noch nicht sagen:** Welche Methoden erfunden oder unbelegt sind. Nicht vorab markieren. Das Arbeitsblatt enthält den Code zum Nachlesen, während der Arbeitsauftrag projiziert wird.

## Folie 19 Was könnt ihr daraus übernehmen

**Sechs Minuten Pair-Arbeit:** Zwei Minuten markieren, zwei Minuten Rückfrage und Prüfungen formulieren, zwei Minuten im Pair abgleichen. Kategorien: gegeben, angenommen, noch zu prüfen.

**Auftrag:** „Welche Informationen fehlen für eine Integration? Formuliert die wichtigste Rückfrage. Welche Belege würdet ihr vor dem Einsatz verlangen? Was ändert die Kennzeichnung hypothetisch?“

**Wenn ein Pair schnell fertig ist:** Nach Rückgabetypen, Fehlerverhalten und Außenwirkungen fragen. Wenn ein Pair hängen bleibt: „Woher wisst ihr, dass diese Methode so aufgerufen werden darf?“

## Folie 20 Auswertung Entwurf und Integration

**Drei Minuten inklusive zweier Beiträge.** Erst Beobachtungen hören, dann die Tabelle zeigen und ergänzen.

**Gegeben:** Name OrderManagementLibrary und das Ziel, eine Bestellung zu prüfen und zu versenden.

**Im Kontext nicht belegt:** Importpfad, Klasse OrderManager, Konstruktor, verify\_order, send\_order, synchrone Ausführung, Datenmodell von order\_data sowie Dictionary-Rückgaben mit success-Schlüssel.

**Wichtige Differenzierung:** Es ist nicht bewiesen, dass diese Methoden in der Realität nicht existieren. Der vorliegende Kontext liefert keinen Nachweis. Als ausdrücklich hypothetischer Entwurf kann der Code nützlich sein. Eine fertige Integration ist damit nicht belegt.

**Geeignete Rückfrage:** „Bitte zeige Importpfad, Typen, Methodensignaturen, Rückgabeverträge und ein funktionierendes Nutzungsbeispiel.“

**Passende Prüfungen:** Dokumentation und Paketversion abgleichen. Integration bauen. Verhalten gegen Stubs beziehungsweise in einer Testumgebung prüfen. Versand kann eine Außenwirkung haben; keine echten Bestellungen senden.

**Übergang:** „Ein gekennzeichnete Entwurf ist anders zu bewerten als eine Ergänzung, die als Tatsache präsentiert wird. Genau das sehen wir im nächsten Beispiel.“

**Beobachtungen der Gruppe:**

---

---

# Halluzination und unabhängige Belege

Minute 43-48 | Folien 21-23

## Folie 21 Kontrastfall Projektupdate

**Zwei Minuten:** Notizen und Ergänzungen vorlesen, dann fragen: „Welche dieser Aussagen stützen die Notizen?“ Erst Antworten abwarten, anschließend auflösen.

**Tatsächlich bekannt:** Maya fragt ein überarbeitetes Angebot an. Das Team bespricht den Startzeitplan erneut, sobald das Angebot vorliegt.

**Unbelegte Ergänzungen:** Maya sei projektverantwortlich, die Spezifikationen seien überholt, der Start verschiebe sich um zwei Wochen. Der ursprüngliche Auftrag unterstellt bereits eine vom Lieferanten verursachte Verzögerung. Das macht die Behauptung nicht wahr.

**Quellentreue Formulierung:** „Maya wird ein überarbeitetes Lieferantenangebot anfragen. Sobald es vorliegt, bespricht das Team den Startzeitplan erneut. Verzögerungsgrund, Projektverantwortung und neuer Starttermin gehen aus den Notizen nicht hervor.“

**Präzise bleiben:** Die Angaben sind durch die Notizen nicht belegt. Wir haben nicht nachgewiesen, dass sie in der Außenwelt falsch sind. Der vorhandene Screenshot dokumentiert eine einzelne Antwort, keinen Modellbenchmark.

## Folie 22 Was wir hier Halluzination nennen

**Eine Minute:** „Wir nennen hier Ausgaben Halluzinationen, die erfundene, falsche oder durch die Quelle nicht gedeckte Angaben als Tatsachen darstellen.“

**Abgrenzen:** Nicht jede schlechte Antwort ist eine Halluzination. Ein missverständlicher Auftrag, eine unpassende Implementierung und ein ausdrücklich hypothetischer Entwurf sind unterschiedlich zu beurteilen.

**Kernpunkt:** Fehlende Informationen führen nicht automatisch zu einer Rückfrage. Besserer Kontext und bessere Anweisungen können helfen, garantieren aber keine korrekte Ausgabe.

## Folie 23 Prüfung braucht unabhängige Belege

**Zwei Minuten:** Jede Behauptung mit einer passenden Prüfung verbinden.

| Behauptung                | Geeigneter Nachweis                       |
|---------------------------|-------------------------------------------|
| Die API existiert         | Offizielle Dokumentation und Paketversion |
| Der Code funktioniert     | Build und passende ausgeführte Tests      |
| Das steht im Repository   | Datei, Fundstelle und Versionsstand       |
| Das erfüllt die Fachregel | Akzeptanzkriterien und fachliche Prüfung  |

**Betonen:** Die Bestätigung durch dasselbe Modell ist kein unabhängiger Nachweis. Quellen öffnen und ihren Inhalt abgleichen. Tests prüfen ausgewähltes Verhalten und beweisen keine vollständige Fehlerfreiheit.

**Übergang bei Minute 48:** „Wir können nicht einfach Korrektheit bestellen. Wir können aber die Arbeitsgrundlage verbessern: Welche relevanten Informationen bekommt das Modell tatsächlich?“

# Kontext und sichere Nutzung

Minute 48-70 | Folien 24-31 | Pause von Minute 60 bis 70

## Folie 24 Prompting und Kontext

**30 Sekunden:** Von der fehlenden Library-Schnittstelle zur Informationsgrundlage überleiten. „Welche Informationen hat das Modell überhaupt, und welche fehlen?“

## Folie 25 Woher kommt die Information

**Zwei Minuten:** Gelernte Parameter, aktueller Kontext, nachgeladene Quellen und gespeicherte Notizen unterscheiden. Das sind Herkunftskategorien, keine vier separaten Speicher im Modell. Quellen und Notizen müssen für den Aufruf in den Kontext gelangen. Produkt-Memory ist eine optionale Anwendungsfunktion.

## Folie 26 Prompting und Context Engineering

**Zwei Minuten:** Anweisung und weitere Informationen im Bild verfolgen. Prompting formuliert die Aufgabe. Context Engineering stellt relevante Informationen zusammen und hält sie bei Bedarf aktuell. Für die Bestellaufgabe gehören Datenmodell, Fachregeln und Prüffälle dazu. Die Anweisung selbst ist Teil des Kontexts.

## Folie 27 Das Context Window ist begrenzt

**Zwei Minuten:** Sichtbarer Chat und tatsächlich gesendeter Kontext können abweichen. Die Anwendung kann auswählen, kürzen, zusammenfassen oder nachladen. Die Budgetanteile im Bild sind schematisch. Input- und Outputgrenzen hängen vom Modell ab. Ein großes Fenster garantiert nicht, dass jedes Detail zuverlässig genutzt wird.

## Folie 28 Relevanz schlägt bloße Menge

**90 Sekunden:** Gute Grundlage: betroffene Funktion, aufgerufene Typen, Fachregeln und reproduzierbarer Fehler. Problematisch: veraltete Dokumentation, widersprüchliche Vorgaben und unpassende Dateien. Bei langen Aufgaben Zwischenstand und offene Annahmen sichern, relevante Quellen gezielt nachladen.

## Folie 29 Welche Daten dürfen wir verwenden

**Eine Minute:** Vor der ersten Live-Modellübung den freigegebenen Zugang und synthetische Daten bestätigen. Secrets gehören nicht in Prompts oder Anhänge. Für echte Kundendaten gelten die Organisationsvorgaben. Keine pauschale Zusage zu Speicherung oder späterem Training geben.

## Folie 30 Fremde Inhalte können Anweisungen enthalten

**Drei Minuten mit Kurzfrage:** README-Inhalt kann versuchen, eine ungewollte Aktion auszulösen. „Darf die Anwendung dem Upload folgen, nur weil die README ihn verlangt?“ Erwartung: Nein. Quelleninhalt erteilt keine Berechtigung. Rechte, Netzwerkzugriffe und Freigaben müssen technisch begrenzt sein. Das Bild zeigt eine gewünschte blockierte Aktion, keine universelle Abwehr. Beispiel ausschließlich analysieren.

## Folie 31 Pause

**Bei Minute 60 stoppen:** Konkrete Rückkehrzeit nennen und aufschreiben. Nach zehn Minuten geht es mit der Bestellübung weiter. Während der Pause keine zusätzlichen Pflichtaufträge verteilen.

# Vergleich mit besserem Kontext

Minute 70-88 | Folien 32-35 | Arbeitsblatt Abschnitt 2

## Folien 32 und 33 Auftrag und gleicher Rahmen

**Eine Minute:** Beide Runden verwenden denselben C#-Basisprompt, dieselben Typen und dieselbe Funktion `Validate(Order? order)`. Nur Runde 2 ergänzt Fachregeln und erwartete Beispiele. Gleicher Modellzugang und gleiche Einstellungen. Neuer Chat für Runde 2, dort Basisprompt und Ergänzung gemeinsam senden.

**Sprechimpuls:** „Wir untersuchen, welche Entscheidungen zunächst Annahmen waren und welche mit klaren Regeln überprüfbar werden. Wir erwarten nicht, dass jede zweite Antwort automatisch besser ist.“

## Folie 32 Arbeitsphasen moderieren

**Minute 71-74:** Basisprompt senden, Antwort speichern, Annahmen markieren. Eine Rückfrage ist ein sinnvolles Ergebnis. Wenn kein Code entsteht, später die erkannten Informationslücken vergleichen.

**Minute 74-79:** Neuer Chat mit demselben Basisprompt plus Fachregeln. Memory, Projektanweisungen und automatische Dateisuche nach Möglichkeit ausschließen, andernfalls als Einfluss notieren.

**Minute 79-85:** Beide Ausgaben anhand der Fälle lesen. Kein verpflichtender Testlauf. „Per Review erfüllt“, „verletzt“ oder „unklar“ festhalten. Keine bestandenen Tests behaupten.

## Folie 34 Erwartete Ergebnisse im Review

Basis: K-1, eine Position mit Menge 1 und Preis 20 EUR, Gesamtbetrag 20 EUR. Jede Änderung ist ein unabhängiger Fall.

| Priorisierter Fall                 | Erwartung in Runde 2                                                     |
|------------------------------------|--------------------------------------------------------------------------|
| Gültige Basis                      | <code>IsValid true</code> , <code>RequiresApproval false</code>          |
| null-Order                         | Fehler, <code>IsValid false</code> , <code>RequiresApproval false</code> |
| TotalAmount 19 statt 20            | Summenfehler, ungültig                                                   |
| Preis und Gesamtbetrag 10000       | Gültig, keine Freigabe                                                   |
| Preis und Gesamtbetrag 10000,01    | Gültig, Freigabe erforderlich                                            |
| Leere Items und Gesamtbetrag 10001 | Ungültig, <code>RequiresApproval false</code>                            |

**Weitere Fälle im Arbeitsblatt:** null/leere Items, null-Position, leere oder Whitespace-Kundennummer, Menge  $\leq 0$ , negativer Preis, decimal-Überlauf. Bei ungültigen Eingaben strukturierter Fehler statt Exception. Fehlercodes und Fehlerreihenfolge bleiben offene Entscheidungen.

## Folie 35 Auswertung

**Minute 85-88:** Zwei Beobachtungen sammeln. „Welche Annahme wurde durch eine Regel ersetzt? Welche bekannte Regel wird verletzt? Was bleibt ungeprüft?“ Unbekannte Regeln nicht rückwirkend als Fehler von Runde 1 zählen.

**Übergang:** „Das LLM kann den Validator entwerfen. Zur Laufzeit prüft klassischer Code die Bestellung. Nun schauen wir, wie ein Assistant bei einer Fehlersuche Informationen und Werkzeuge verbindet.“

# RAG Tools und Agenten

Minute 88-96 | Folien 36-41 | Ein durchgehendes Entwicklerbeispiel

## Folien 36 und 37 Eine abgelehnte Bestellung untersuchen

**Eine Minute:** „Eine gültige Bestellung scheitert in einer unbekannten Codebasis. Was braucht der Assistant, um die Ursache zu untersuchen?“ Kurzen Zuruf sammeln: Validator, Fachregeln, reproduzierbares Beispiel und Tests. Keine Live-Demo erforderlich.

## Folie 38 RAG Quellen als Kontext

**Zwei Minuten:** RAG heißt Retrieval-Augmented Generation. Die Suche findet zum Beispiel OrderValidator.cs, Tests und Fachregeln. Relevante Auszüge gelangen zusammen mit der Frage in den Modellkontext. Das Modell formuliert daraus eine Erklärung mit Fundstellen.

**Am Bild erklären:** Quellen ergänzen den Kontext; das Modell wird dabei nicht neu trainiert. Suche kann Text, Symbole oder semantische Ähnlichkeit verwenden. Eine Vektordatenbank ist nicht für jede Repository-Suche erforderlich. Retrieval kann als Tool implementiert sein.

**Grenze:** Die Suche kann eine entscheidende Datei übersehen. Die Antwort kann die gefundene Quelle falsch interpretieren. Fundstelle und Stützung prüfen; Abdeckung separat beurteilen.

## Folie 39 Tool Calling Wer führt aus

**Zwei Minuten:** Die fünf Schritte am Bild verfolgen: Modell schlägt runTests vor. Anwendung prüft Argumente, Rechte und Umgebung. Tool führt Tests aus. Ergebnis oder Fehler gelangt zurück. Modell wertet den tatsächlichen Befund aus.

**Betonen:** Der Toolname ist ein Beispiel. „Ich habe getestet“ braucht einen tatsächlichen Ausführungsnachweis. Gültiges JSON ist keine Berechtigung. Auch Tests führen Code aus und gehören in eine freigegebene Umgebung.

## Folie 40 Agenten Schritte mit Rückkopplung

**Zwei Minuten:** „Nach dem Lesen einer Datei wählt das Modell den nächsten Schritt. Es kann Tests anfordern, den Befund auswerten und weitere Dateien lesen.“ Damit entsteht eine Schleife mit Rückkopplung.

**Arbeitsdefinition:** Ein Agent kann innerhalb kontrollierter Grenzen nächste Schritte wählen. Eine feste Pipeline legt Schritte im Voraus fest. Ein einzelner Toolaufruf ist noch keine solche Schleife.

**Grenzen:** Rechte, Zeit, Kosten und Schrittlimit muss die Anwendung durchsetzen. Rückfrage, fehlende Freigabe oder erreichtes Limit können den Ablauf stoppen. Eine Modellbewertung allein beweist keinen Erfolg.

## Folie 41 Wo klassische Software die Grenze setzt

**Eine Minute:** Summen, Schema-Prüfung, Berechtigungen und kontrollierte Pipelines technisch umsetzen. Das Modell kann bei Entwurf und Erklärung helfen. Verbindliche Regeln dürfen nicht ausschließlich als Bitte im Prompt existieren.

**Übergang bei Minute 96:** „Jetzt verteilt ihr selbst die Verantwortung: Was darf das Modell entscheiden und was braucht eine technische oder menschliche Kontrolle?“

# Gruppenarbeit und Auswertung

Minute 96-108 | Folien 42-44 | Arbeitsblatt Abschnitt 3

## Folien 42 und 43 Wer übernimmt welchen Teil

**Zwei Szenarien pro Gruppe:** A: unbekannte Codebasis erklären und eine Fehlerursache prüfen. B: Produktionsdeployment vorbereiten und auslösen. Datenquelle, Modellaufgabe, Tool oder Code, Prüfung und Freigabe benennen.

**Acht Minuten:** Eine Minute individuell nachdenken, fünf Minuten gemeinsam entscheiden, zwei Minuten einen strittigen Punkt für das Plenum vorbereiten. Folie 43 während der Arbeit stehen lassen. Keine Implementierung verlangen.

**Beim Herumgehen fragen:** „Woher kommt der Beleg? Wer prüft die Berechtigung? Was passiert, wenn Daten fehlen oder widersprüchlich sind?“ Bewertet wird die Begründung, nicht die Anzahl genannter Bausteine.

## Folie 44 Auswertung Erklärung und Aktion

**Vier Minuten ab Minute 104:** Zwei Beiträge hören, dann ergänzen. Pro Gruppe nur einen strittigen Punkt behandeln.

**Codebasis verstehen:** Retrieval beziehungsweise Lesetools liefern Dateien. Das Modell erklärt und bildet Hypothesen. Builds und passende Tests prüfen beobachtbares Verhalten. Dateiverweise und Versionsstand abgleichen. Eine gefundene Datei beweist keine vollständige Analyse.

**Deployment auslösen:** Das Modell unterstützt Planung. Eine kontrollierte Pipeline führt aus. Zielumgebung, Revision, Checks, Rechte, Freigaben nach tatsächlichem Prozess und Rücksetzplan prüfen. Menschen verantworten Entscheidungen und Ausnahmen.

**Keinen universellen Prozess behaupten:** Freigaben hängen von der Organisation und Aufgabe ab. Ein vorhandener Freigabeschritt ersetzt keine technischen Rechteprüfungen. Bei fehlenden Belegen nachladen, nachfragen oder stoppen.

## Reservefälle bei anderem Gruppenbedarf

Einen Fall ersetzen, niemals zusätzliche Szenarien in dieselbe Übungszeit pressen.

| Szenario                  | Erwartete Aufteilung und Grenze                                                                                               |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Resturlaub                | Tool liest aktuelles Fachsystem. Identität, Berechtigung und Stichtag prüfen. Richtlinie allein liefert keinen Kontostand.    |
| Rechnungssumme            | Code rechnet nach definierten Regeln. Vom Modell extrahierte Eingabewerte vor der Berechnung prüfen.                          |
| Supportticket             | Modell schlägt Kategorie vor. Erlaubte Werte, Qualität und Stichproben prüfen.                                                |
| 300 Dokumente durchsuchen | Retrieval findet Quellen. Modell extrahiert. Version und Abdeckung separat prüfen; vollständige Verarbeitung kann nötig sein. |

**Übergang bei Minute 108:** „Wir haben die Bausteine und ihre Grenzen kennengelernt. Zum Abschluss prüfen wir, welche Erklärungen und konkreten Änderungen ihr mitnehmt.“

# Wissenscheck und Transfer

Minute 108-120 | Folien 45-48

## Folie 45 Kurzer Wissenscheck

**Drei Minuten:** Zu jeder Frage eine Begründung hören. Bei knapper Zeit Antworten reihum kurz sammeln, keine neue Grundsatzdiskussion eröffnen.

- 1. Eine Datei mitgeben:** Ihr Inhalt kann Teil des aktuellen Kontexts werden. Die Parameter bleiben bei normaler Inference unverändert. Sichtbarkeit in der Oberfläche garantiert keine vollständige Verarbeitung. Auf Auswahl und Zusammenfassung durch die Anwendung verweisen.
- 2. Dreimal identische Antwort:** Beobachtete Wiederholbarkeit ist belegt. Richtigkeit folgt daraus nicht. Ein stabiler Fehler bleibt ein Fehler.
- 3. RAG-Antwort mit Quellen:** Prüfen, ob Quelle und Fundstelle existieren, aktuell und relevant sind und die konkrete Aussage stützen. Die Vollständigkeit der Suche bleibt separat zu beurteilen.
- 4. README fordert Upload und JSON ist gültig:** Keine automatische Berechtigung. Quelleninhalt autorisiert keine Aktion. Auftrag, Herkunft, Semantik, Rechte und gegebenenfalls Freigabe prüfen.

## Folie 46 Fünf Kernbotschaften

**Eine Minute:** Eine Aussage mit einem „Hilft / Nervt“-Beispiel vom Einstieg verbinden. Dann die fünf Punkte verdichten: Assistant verbindet Modell, Kontext und Tools; Fähigkeiten garantieren keine Korrektheit; gute Quellen und Anforderungen helfen; Software setzt Grenzen durch; überprüfbare Belege tragen die Entscheidung.

## Folie 47 Mein Transfer

**Fünf Minuten:** Zwei Minuten allein schreiben, drei Minuten im Pair besprechen. Jede Person hält eine Erkenntnis, eine konkrete Änderung und eine offene Frage fest.

**Hilfreiche Nachfrage:** „Bei welcher nächsten Aufgabe setzt du das ein, welchen Kontext gibst du dazu und wie prüfst du das Ergebnis?“ Ein konkreter nächster Versuch ist hilfreicher als „Ich werde vorsichtiger sein“.

**Danach zwei Minuten offene Fragen sammeln:** Fragen einem Folgemodul zuordnen. Nicht spontan einen zusätzlichen Vortrag starten. Offene organisatorische Fragen separat notieren.

## Folie 48 AI Assisted Coding

**Letzte Minute:** „Im nächsten Modul nutzen wir dieses Verständnis praktisch: Aufgaben zerlegen, relevanten Codekontext bereitstellen und Änderungen in kleinen, überprüfbaren Schritten umsetzen.“

**Einordnung der Folgemodule:** Modul 2 kontrollierte Coding-Workflows. Modul 3 Tests und Feedback. Modul 4 Verhaltenserhalt beim Refactoring. Modul 5 Spezifikationen für größere Aufgaben. Modul 6 eigene Agenten mit Tools und Grenzen.

**Zum Abschluss:** Auf die Verantwortung für den übernommenen Code verweisen. Bei Minute 120 schließen.

**Offene Fragen für die nächsten Module:**

---

---

# Rückfragen und fachliche Präzisierungen

Referenz für die Vorbereitung und bei Fragen im Training

## Lernt das Modell aus unserem Chat

Normalerweise verändert die Antwortgenerierung keine Modellparameter. Verlauf oder gespeicherte Notizen können in spätere Aufrufe gelangen und so wie Erinnern wirken. Speicherung und mögliche spätere Datennutzung sind davon getrennte Produkt- und Vertragsfragen. Keine pauschale Zusage machen.

## Wenn es Tokens vorhersagt wie kann es programmieren

Die Token-Erzeugung beschreibt, wie Ausgabe entsteht. Gelernte Parameter können komplexe Muster und Zusammenhänge repräsentieren. Nachtraining formt das Verhalten zusätzlich. Aus dem Mechanismus folgt weder Unfähigkeit noch garantierte Korrektheit. Die konkrete Lösung bleibt zu prüfen.

## Verhindern guter Kontext oder Reasoning die Fehler

Sie können die Arbeitsgrundlage beziehungsweise Bearbeitung verbessern. Daraus folgt keine Garantie. Quellen können unvollständig oder widersprüchlich sein und das Modell kann Anforderungen falsch umsetzen. Angezeigte Herleitungen ersetzen keinen unabhängigen Nachweis.

## Ist RAG dasselbe wie Fine Tuning

Nein. Bei RAG gelangen gefundene Quellen in den Kontext. Training beziehungsweise Fine-Tuning verändert Modellparameter. Für diese Schulung genügt die Unterscheidung zwischen Kontext bereitstellen und Parameter anpassen. Die Umsetzung von Fine-Tuning ist kein Lernziel.

## Kann das Modell seine Antwort selbst prüfen

Eine zweite Prüfung kann Fehler finden. Eine bloße Selbstbestätigung liefert jedoch keinen unabhängigen Beleg. Bei Code nach konkreten Fundstellen, Builds, Testausgaben und erfüllten Akzeptanzkriterien fragen.

## Ist jede Anweisung in einer README ein Angriff

Nein. Projektdateien können hilfreiche Arbeitsanweisungen enthalten. Sie erhalten dadurch aber keine unbegrenzte Autorität. Entscheidend sind Herkunft, Nutzerauftrag, geltende Vorgaben und technisch erlaubte Aktionen. Das Beispiel illustriert eine mögliche Manipulation, keine pauschale Bewertung von READMEs.

## Was tun wenn ein Modell keine Fehler macht

Das ist ein gutes Ergebnis. Keine Halluzination erzwingen. In Runde 1 gute Rückfragen und korrekt benannte Unsicherheit würdigen. In Runde 2 untersuchen, welche Informationen geholfen haben und welche Prüfungen trotzdem nötig bleiben.

## Grundlage dieser Referenz

Abgeglichen mit module-1-ai-basics.marp.md, arbeitsblatt-modul-1.md, Trainer/modul-1-leitfaden.md und Curriculum.md. Fachliche Vertiefungen stehen in den Quellen der Präsentation. Formulierungshilfen und Zeitanker dienen der Moderation, nicht als zusätzliche Lerninhalte.

## Eigene Ergänzungen:

---

---